

CS3601 操作系统

测验试卷参考答案

1. 以下为寄存器的初始值及按顺序执行的指令，请将表格补充完整。（前序指令的执行结果会影响后续指令执行）（5 分）

寄存器	初始值
X0	0xffffffff00000000
X1	0x0
X2	0x0

指令	目标寄存器	结果
asr x0, x0, #0x11		
mvn x1, x1		
eor x2, x0, x1		
add x2, x0, x2		
lsr x2, x2, #6		

解:

指令	目标寄存器	结果
asr x0, x0, #0x11	X0	0xffffffffffff8000
mvn x1, x1	X1	0xfffffffffffffff
eor x2, x0, x1	X2	0x0000000000007fff
add x2, x0, x2	X2	0xfffffffffffffff
lsr x2, x2, #6	X2	0x03ffffffffffff

2. 不使用栈，将以下函数转换成 ARM64 汇编。（5 分）

（参数 x 通过 x0 寄存器传参；函数返回结果需放在 x0 寄存器中；无需考虑函数调用过程中的寄存器保存/恢复等栈相关操作）

```
int64_t sum(int64_t x) {  
    int64_t ret = 0;  
    for (int64_t i = 1; i <= x; i++) {  
        ret += x;  
    }  
    return ret;  
}
```

解:

```
sum:
    mov x1, #0
    mov x2, #1

loop_start:
    cmp x2, x0
    bgt loop_end
    add x1, x1, x0
    add x2, x2, #1
    b    loop_start

loop_end:
    mov x0, x1
    ret
```

3. 阅读以下汇编函数，回答问题：

```
foo:
    stp x19, x20, [sp, #-16]!
    mov x19, #0
    mov x20, #0

loop:
    cmp x20, x2
    bge done

    ldr x5, [x0, x20, lsl #3]
    ldr x6, [x1, x20, lsl #3]

    mul x5, x5, x6
    add x19, x19, x5

    add x20, x20, #1

    b    loop

done:
    mov x0, x19
    ldp x19, x20, [sp], #16
    ret
```

- 1) 请分析函数 foo 的作用，参数个数，每个参数的意义。(3 分)
- 2) 为什么使用 x19 和 x20 寄存器的时候需要保存在栈上，而其他寄存器不需要?(2 分)

解:

- 1) 该函数计算两个 64 位整型数组（向量）的点积。它将两个数组对应位置的元素相乘，然后将所有乘积累加求和。有 3 个参数：x0 指向第一个 64 位整型数组的基地址；x1 指向第二个 64 位整型数组的基地址；x2 是 64 位整数，表示数组的长度。
- 2) 在 ARM64 中，x19 和 x20 属于被调用者保存的寄存器，如果一个函数要使用它们，就必须在使用前将它们的原始值保存到栈上，并在函数返回前恢复，以确保不破坏调用者函数的上下文；而函数中使用的 x5、x6 等寄存器属于调用者保存的寄存器，被调用函数可以自由使用它们而无需负责保存和恢复。

4. 局部变量在栈上的顺序和函数调用参数入栈的顺序分别是什么？原因分别是什么？(4 分)

解: 局部变量在栈上的存储顺序主要**由编译器决定**，其原因主要是为了满足数据对齐要求和进行编译优化。为了保证访问效率，多字节的数据类型需要存放在特定边界的地址上（如 8 字节对齐），编译器会据此调整变量顺序，因此其在内存中的顺序不一定与代码中的声明顺序一致。

函数调用参数的入栈顺序为**从右至左**，即函数的最后一个参数最先入栈，位于栈上最高地址；第九个参数最后入栈，位于栈上最低地址。这样设计的主要原因是为了支持可变参数函数（如 printf）。被调用函数总能从一个相对于栈指针已知的栈地址找到第一个栈上传递的参数（即第九个参数），然后根据该参数来解析后续参数的数量和类型，从而依次向高地址方向遍历栈来获取所有参数。

5. 在函数调用过程中：

- 1) X0 寄存器是调用者保存还是被调用者保存？为什么？(2 分)
- 2) 一个函数如果想要将多个变量作为返回值应该怎么办？给出两种办法。(2 分)

解:

- 1) X0 寄存器是**调用者保存**的。X0 寄存器既用于传递函数的第一个参数，也用于存放函数的返回值。被调用函数使用 X0 来返回其计算结果必然会覆盖调用者传入的原始参数值，因此，被调用函数无需保留 X0 的原始值。如果调用者在函数调用返回后仍需要使用传入 X0 的原始值，则调用者自身有责任在调用前将其保存。
- 2)
 - 方法一：通过指针参数返回。函数可以接收一个或多个指针作为其输入参数，并将需要返回的多个值写入这些指针所指向的内存地址中。当函数返回后，调用者可以通过这些指针访问到所有的返回结果。
 - 方法二：返回一个结构体。函数可以将所有需要返回的变量封装在一个结构体中，并将这个结构体作为函数的返回值类型。如果结构体较小，其内容可以直接通过寄

寄存器返回；如果结构体较大，会由调用者分配内存，并将该内存的地址作为隐式参数传递给被调用函数，被调用函数随后将返回数据填充到该内存区域。

6. 特权级切换时已有硬件保存上下文，为什么还需要软件保存/恢复上下文？给出两个理由。（4分）

解：

- 理由一：硬件保存的上下文不完整。硬件仅保存了程序计数器（ELR_EL1）、处理器状态（SPSR_EL1）等关键寄存器，但并不会保存通用寄存器（X0-X30）或浮点寄存器。内核的异常处理程序本身也需要使用这些寄存器来进行计算、寻址等操作。如果软件不先将用户进程的这些寄存器值保存到栈或内存中，它们就会被内核代码覆盖，导致当返回用户态时，用户进程的执行状态已被破坏，从而引发程序崩溃或错误。
- 理由二：为了实现进程的调度与切换。一个中断或系统调用可能触发操作系统调度器的工作，决定暂停当前进程（进程 A），转而去执行另一个就绪的进程（进程 B）。在这种情况下，操作系统软件必须负责将进程 A 的完整执行上下文（包括所有通用寄存器、程序计数器、栈指针等）从 CPU 中保存到你位于内存的进程控制块（PCB）中，然后再从进程 B 的 PCB 中加载其之前被保存的完整上下文到 CPU 中，从而完成进程的切换。这个保存和恢复完整上下文以实现多任务并发执行的逻辑，完全超出了硬件设计的范畴，必须由软件来管理。

7. 操作系统通过系统调用允许应用调用部分系统能力，请回答以下问题：

- (1) 简述 Flex-SC 执行一个系统调用的流程。（3 分）
- (2) 适合用 Flex-SC 优化的系统调用有哪些特点？至少写出两种。（2 分）
- (3) 什么样的系统调用不适合使用 Flex-SC 优化？给出两种类型。（2 分）
- (4) vDSO 通过将系统调用逻辑放入用户态进行系统调用加速，请分别给出 1 个适合与不适合用 vDSO 加速的系统调用，并分别说明原因。（5 分）

解：

- (1) 应用程序通过一个特殊系统调用向内核注册一块共享内存区域作为请求页。当应用程序中的线程需要发起系统调用时，它将系统调用编号及其参数打包成一个请求，并将其填入该共享请求页的一个空闲槽位中。当请求页中积累了足够的请求时，该线程会执行一次真正的系统调用，以此通知内核处理这批被暂存的请求。收到通知后，内核中的专用工作线程会被唤醒，它们直接访问该共享内存页，从中读取所有待处理的请求并依次在内核态下执行。每个请求执行完毕后，其结果会被写回到请求页中对应的槽位。最终，在所有批处理请求都执行完毕后，内核会通知用户态的应用程序，唤醒那些等待结果的线程，这些线程随后便可以从共享页中安全地读取各自的返回值并继续执行。
- (2)
 - 内核执行时间极短：这类系统调用的主要开销在于进出内核的上下文切换，而非其本身在内核中的计算。例如 `getpid()` 或对已在页缓存中的文件进行的小数据量 `read/write`。Flex-SC 通过批处理显著降低了单位调用的上下文切换开销。

- 调用频率非常高：对于在紧密循环中大量调用系统调用的应用，如高性能网络服务器或数据库，Flex-SC 能够将大量的调用请求捆绑在一起，最大化分摊效应，从而获得显著的性能提升。
- (3)
- 长时间阻塞的系统调用：例如 `wait()`, `poll()`, `select()` 或从慢速设备读取数据的 `read()`。如果批处理中包含一个此类调用，它将阻塞整个批次的处理，导致其他所有非阻塞的调用也必须等待，从而严重影响性能和响应时间。
 - 改变进程核心状态或控制流的系统调用：例如 `fork()`, `execve()`, `exit()`。这类调用的语义非常复杂，它们会创建新进程、替换进程镜像或终止进程，无法被简单地封装在“请求-响应”的批处理模型中。
- (4)
- 适合的系统调用：`gettimeofday`。原因是：这类系统调用的核心任务是读取内核中维护的、对所有进程共享的时间信息。这些信息可以被安全地以只读方式暴露给用户态。vDSO 将读取这些内核变量并进行计算的少量代码映射到用户空间，使得调用完全在用户态进行，无需任何上下文切换，从而极大地提升了频繁获取时间的应用的性能。
 - 不适合的系统调用：`open`（或 `write`）。原因是：这类系统调用涉及核心的系统安全与资源管理，必须在拥有特权的内核态执行。例如，`open` 需要进行文件系统路径解析和权限检查，如果将这些逻辑放入用户态，就意味着任何应用程序都可以绕过操作系统的安全模型，随意访问磁盘上的任何文件或直接操作硬件，这将导致整个系统安全体系的崩溃。因此，它们必须通过陷入内核的方式来执行。

8. 阅读下面的 C 代码，回答问题：

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int counter = 1;

int main() {
    int status = 0;
    counter++;
    if (fork() == 0) {
        counter--;
        printf("%d\n", counter);
        return 0;
    }
    counter++;
    waitpid(-1, &status, 0);
    counter++;
    printf("%d\n", counter);
    return 0;
}
```

```
}
```

- 1) 请写出上述程序所有可能的输出结果。(2 分)
- 2) 将上述程序编译为 a.out, 在 Linux shell 执行 ./a.out 运行程序。请从 shell 视角介绍, 程序是如何被加载的? (哪个进程调用哪个系统调用, 每个系统调用的功能/目的是什么) (4 分)
- 3) 上述程序运行起来后, 何时会发生用户-内核上下文切换? 请给出至少两种情况。(2 分)

解:

- 1) 该程序唯一的输出结果为:

```
1
4
```

- 2)
 - 首先, shell 进程调用 fork() 系统调用, 其目的是创建一个新的子进程, 该子进程是 shell 进程的一个副本。
 - 接着, 在这个新创建的子进程中, 会立即调用 execve() 系统调用。execve() 的功能是根据指定的文件名 (./a.out), 将新的程序加载到当前进程的内存空间中, 替换掉原有的 shell 副本的程序映像。
 - 与此同时, 原来的父进程 (即 shell) 会调用 waitpid() 系统调用, 其目的是挂起自身, 等待子进程 (即运行 a.out 的进程) 执行结束, 以便在子进程结束后回收其资源并重新显示命令提示符。
- 3)
 - 执行系统调用时: 当程序需要请求操作系统内核提供的服务时, 会通过 svc 指令主动触发从用户态到内核态的切换。在此代码中, 调用 C 库函数 fork()、waitpid() 以及 printf() 时, 都会发生这种切换。
 - 发生硬件中断时: 硬件中断会强制中断当前正在用户态执行的程序, 使 CPU 控制权转移给内核中的中断处理程序。例如发生时钟中断时, 操作系统调度器会获得 CPU 控制权, 以便进行进程调度决策。

9. 阅读下面的 C 代码, 回答问题:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    for (int i = 0; i < 2; i++) {
        fork();
        printf("A\n");
    }
}
```

```
    return 0;
}
```

- 1) 上面这段程序会输出几个 A? 为什么? (3 分)
- 2) 如果将上述代码的 `printf("A\n")` 改为 `printf("A")`, 程序会输出几个 A? 为什么? (提示: 默认情况下, `printf` 的输出内容会缓冲在内存中, 不会立即输出到屏幕, 直到遇到换行符、显式调用 `fflush` 或程序退出) (3 分)

解:

- 1) **6 个**。其原因是: 在 `i=0` 的第一次循环中, 原始进程创建了一个子进程, 此时总共有两个进程。这两个进程都会继续执行 `printf` 语句, 因此输出了 2 个 A。在 `i=1` 的第二次循环开始时, 已有的这两个进程各自都会再次调用 `fork()`, 分别创建自己的子进程, 使得进程总数变为四个。这四个进程随后都会执行 `printf` 语句, 因此又输出了 4 个 A。
- 2) **8 个**。当 `printf("A")` 执行时, 由于没有换行符, 输出的字符 A 会被暂存在内存缓冲区中。当 `fork()` 系统调用发生时, 它会复制父进程的整个地址空间, 其中就包括了 this 尚未被清空的 I/O 缓冲区。

具体流程如下: 在第一次循环中, 原始进程首先调用 `fork()` 创建子进程, 这两个进程各自执行 `printf("A")`, 将一个 A 存入自己的缓冲区。进入第二次循环时, 这两个进程各自再次调用 `fork()`, 创建出的新子进程会继承父进程此刻的内存状态, 即继承包含一个 A 的缓冲区, 此时进程总数变为四个。最后, 这四个进程都执行 `printf("A")`, 在各自的缓冲区末尾追加一个 A, 从而每个进程的缓冲区中都存有 2 个 A。当进程退出时, 缓冲区被自动清空并输出, 总计输出 $4 * 2 = 8$ 个 A。

10. 阅读下面的 C 代码, 回答问题:

```
#include <stdlib.h>
#include <stdio.h>

int a;
char b[] = "hello";
const char* c;
int const d = 2;

void foo() {
    c = "hello world";
    printf("a: %d\n", a);
    printf("b: %s\n", b);
    printf("c: %s\n", c);
    printf("d: %d\n", d);
}
```

```
int main() {
    foo();
    return 0;
}
```

- 1) 上述代码编译为 ELF 格式文件，a，b，c，d 分别位于 ELF 文件的哪个节中？（4 分）
- 2) printf(“a:%d\n”, a) 的输出结果是什么？为什么？（3 分）
- 3) 上述代码并没有实现 printf 函数，为什么可以在运行时调用到 printf 函数？（2 分）
- 4) 为什么 Linux 进程地址空间需要区分代码段和数据段？（4 分）

解:

- 1) 全局未初始化变量 a 位于 **.bss** 节。全局初始化数组 b 位于 **.data** 节。全局未初始化指针 c 位于 **.bss** 节。全局常量 d 位于 **.rodata** 节。
- 2) 输出“a: 0”。原因是：a 是一个全局未初始化的变量。根据 C 语言标准，全局变量如果在定义时没有被显式初始化，都会被默认初始化为零。当操作系统通过 **execve** 加载并执行该程序时，内核会为.bss 节所描述的区域分配内存，并保证在程序执行前，这块内存区域的所有字节都被清零。在 ELF 加载过程中，位于.bss 节的 a 所在的内存区域会被加载器清零。
- 3) C 语言的动态链接机制。代码中 `#include <stdio.h>` 仅提供了 printf 的函数声明。在链接阶段，链接器将 printf 标记为一个未解析的外部符号，并记录程序对 C 标准库 (libc) 的依赖。当程序运行时，操作系统的动态加载器会先将 C 标准库加载到进程的地址空间中，然后再解析程序中对 printf 的引用，将其指向库中 printf 函数的实际内存地址。
- 4)
 - 安全性：操作系统可以为不同的段设置不同的访问权限。代码段通常被设置为只读和可执行，防止程序在运行中被意外或恶意地修改；而数据段被设置为可读写但不可执行，这可以防止攻击者注入并执行恶意代码。
 - 资源共享和内存效率：代码段的内容对于运行同一程序的所有进程来说是相同的，因此可以被映射到同一块物理内存中，供多个进程共享，从而节省系统内存。
 - 有利于提升 CPU 缓存效率：因为现代处理器通常拥有独立的指令缓存和数据缓存，将两者分开存放可以减少缓存冲突，提高程序执行速度。

11. 同学们在学习了操作系统课程后，对写代码产生了浓厚的兴趣，编写了各种各样的程序。

- 1) 在 linux 系统中运行如下程序，会导致电脑失去响应吗？为什么？如果会，有什么办法防范这种恶意程序？（3 分）

```
int main() {
    while (1);
}
```



```
        return 0;
    }
```

- 2) 在 linux 系统中运行如下程序，会导致电脑失去响应吗？为什么？如果会，有什么办法防范这种恶意程序？（3 分）

```
#include <unistd.h>

int main() {
    while (1)
        fork();
    return 0;
}
```

- 3) 有同学在 Linux 系统中运行如下程序，一段时间后发现电脑变慢了，他 Ctrl+C 终止该程序后，发现电脑很快又恢复正常。请写出电脑变慢和恢复正常的具体原因。（4 分）

```
#include <unistd.h>
#include <stdlib.h>

int main() {
    while (1) {
        if (fork() == 0) {
            exit(0);
        }
        sleep(1); // 程序暂停1秒后继续执行
    }
    return 0;
}
```

解:

- 1) **不会**。这段代码会使单个进程陷入死循环，从而占满一个 CPU 核心的全部计算资源。但由于 Linux 采用了抢占式多任务的内核调度器，操作系统会通过硬件时钟中断定期剥夺该进程的 CPU 使用权，确保其他系统进程和用户应用依然有机会在其他核心上运行。因此，系统整体上仍会保持响应。
- 2) **会**。这段代码会无限循环地创建子进程，导致系统中进程的数量呈指数级增长。每个进程都需要消耗一定的内存和内核资源，进程数量的爆炸式增长会迅速耗尽所有物理内存和交换空间，导致系统因资源枯竭而完全卡死。

防范办法是：限制单个用户能够创建的进程总数。管理员可以为用户设置一个合理的进程数量上限，这样 fork 炸弹在达到该限制后就会因创建进程失败而失效，从而保护系统免于崩溃。

3) 电脑变慢的原因是: 程序循环地创建子进程, 子进程在创建后立即退出。然而, 父进程没有调用 `wait()` 或 `waitpid()` 来回收子进程的资源, 这导致所有退出的子进程都变成了**僵尸进程**。僵尸进程本身不占用内存, 但它在内核的进程表中仍然占据一个条目。由于程序不断地创建僵尸进程, 最终会逐渐耗尽进程表中的空闲位置, 导致系统内任何需要创建新进程的操作都会变得极其缓慢, 整个电脑变得卡顿。

恢复正常的原因是: 当这个父进程被终止后, 它所产生的所有僵尸进程都会变成孤儿进程, 并立即被系统的 `init` 进程回, 释放它们在进程表中占用的条目。进程表恢复正常后, 系统便可以顺利地创建新进程, 性能也随之恢复。

12. 在 AArch64 中, 如何根据页表项判断 PFN 是否指向大页? (3 分)

解: 0 级、1 级、2 级页表项的第 1 位为 0, 表示该页表项是块描述符, 指向大页。

13. 有一个 64 位系统, 支持 5 级页表, 每级页表均支持大页。其虚拟地址索引和偏移量位域如下所示:

虚拟地址索引	偏移量位域
第 1 级	56 ~ 48
第 2 级	47 ~ 39
第 3 级	38 ~ 30
第 4 级	29 ~ 21
第 5 级	20 ~ 12
页偏移量	11 ~ 0

- 1) 翻译一个虚拟地址所需要的页表大小是多少? (3 分) 5 级页表架构能够表示的地址范围是多少? (3 分) 第 1 级页表项指向的大页的大小为多少? (3 分)
- 2) 如果我们希望引入一个 1G 的大页, 那么我们应当在哪一级页表引入块描述符? (3 分)
- 3) 为描述 1536MB 的空间, 如果只采用 4KB 页面, 那么最少需要多少个页表页? (4 分) 如果允许使用大页, 那么最少需要多少个页表页? (3 分)
- 4) 若某 TLB 中最多缓存 64 个 TLB Entry。在每个 Entry 都 valid 的情况下, 请计算最多有多大的地址空间的地址翻译可以被缓存? (3 分) 最少有多大的地址空间的地址翻译被缓存? (3 分)

解:

- 1) 20 KB; 2^{57} B, 即 128 PB; 2^{48} B, 即 256 TB。
- 2) 第 3 级。
- 3) 773 个; 4 个。
- 4) 16 PB; 256 KB。

14. 64 位操作系统中，OS 往往在启动时通过直接映射（Direct Mapping）把所有物理内存映射到内核虚拟地址空间，请回答：页表直接映射的好处是什么？（6 分）

解：

- 简化了内核对物理内存的管理和访问：直接映射为内核提供了一个永久且可预测的虚拟地址来访问任意物理地址。这使得内核仅通过简单的算术偏移就能在物理地址和内核虚拟地址之间转换，极大地简化了内核内存管理子系统（如伙伴系统、Slab 分配器）的实现。
- 提升了内核的运行性能。这种静态的、一一对应的映射关系避免了为内核临时访问内存而动态创建和销毁页表项所带来的高昂开销。地址转换可以被高效地缓存在 TLB 中，从而最大限度地减少了耗时的页表遍历，为内核访问任何物理内存都提供了一条稳定高速的路径。
- 方便与硬件设备交互。它简化了内核与使用直接内存访问（DMA）的硬件之间的交互。DMA 设备操作的是物理地址，当设备完成操作并触发中断时，内核可以立即将硬件报告的物理地址转换为内核虚拟地址来访问数据，无需任何额外的映射步骤。这降低了中断处理的延迟并简化了设备驱动程序的开发。

15. 请回答下列问题：

- 1) AArch64 采用基于 4 级页表的虚拟内存机制，每个页表项大小为 4KB，包含 512 个页表项，支持 4KB、2MB、1GB 的页面大小。请问如果考虑全部使用 2M 大页，翻译 3000 个 2M 虚拟页所需要的最小页表大小是多少？（2 分）
- 2) Linux 操作系统利用 VMA 结构体记录进程已分配的虚拟内存区域，请问：应该使用什么数据结构组织管理 VMA，并简单阐述原因（2 分）
- 3) 进程创建时，操作系统会为用户栈空间分配虚拟内存区，Linux 初始栈大小通常为 8MB。然而，栈空间无法一次性完全分配，其大小在运行时动态增长（例如递归调用或分配大量的局部变量），可能会超出初始栈的空间，请问：操作系统如何在运行时检测栈空间增长的需求并管理相应的 VMA？（3 分，言之有理即可）
- 4) 如果应用程序访问虚拟地址时触发缺页异常，请问可能是由哪些原因造成的？操作系统如何区分不同的原因？（6 分）

解：

- 1) 32 KB。
- 2) 红黑树。原因是：红黑树是一种自平衡的二叉搜索树，它能为关键操作（如根据地址查找对应的 VMA、插入新的 VMA、以及删除 VMA）提供高效且稳定的对数时间复杂度，这对于拥有大量离散内存区域的复杂进程的性能至关重要。
- 3) 操作系统通过处理缺页异常来检测并满足栈的增长需求。当程序访问一个位于当前栈项之下、但尚未分配物理页面的虚拟地址时，会触发一个缺页异常。内核的异常处理程序

会检查到该地址位于栈 VMA 的合法扩展区域内，便认为这是一次合法的栈增长请求。于是，内核会分配一个新的物理页面，建立相应的页表映射，然后返回用户态程序继续执行。如果访问地址与当前栈顶的距离过大，则被视为非法访问，并发送段错误信号。

4) 可能的原因包括：

- 合法请求物理内存：如首次访问代码/数据页、栈空间自动增长、写时拷贝。
- 访问被换出的页：访问的页面之前存在于物理内存，但被操作系统交换到磁盘上。
- 非法访问：访问一个未分配的虚拟地址（段错误）。
- 权限错误：对一个合法地址进行了不允许的操作，如尝试写入一个只读的代码段。

当缺页异常发生时，CPU 会将出错的虚拟地址和异常原因保存到特定寄存器中。内核的异常处理程序首先会检查该出错地址是否存在于进程的某个 VMA 中。如果地址不在任何 VMA 内，则判定为非法访问。如果地址在 VMA 内，内核会检查 VMA 的权限。如果权限也合法，说明这是一个合法请求，内核会进一步检查页表项，判断页面是在交换空间还是需要首次分配，然后执行相应的加载或分配操作。

16. 请回答下列问题：

1) 在执行第 3 行代码时，OS 如何判断应用程序访问的是非法虚拟地址？（3 分）

```
1  int main() {  
2      char *ptr = (char *) 0x0;  
3      *ptr = 'a';  
4  }
```

- 2) 在执行 fork 系统调用后，子进程会复制一整份父进程的地址空间，操作系统利用写时拷贝 (copy-on-write) 实现 fork 性能加速，请阐述如何实现写时拷贝（3 分）
- 3) 换页机制 (swapping) 使我们能够突破物理内存容量限制，把磁盘空间当成内存空间使用，但这也使得缺页异常 + 磁盘操作导致访问延迟增加。请问如何降低换页机制带来的访问延迟开销？（3 分）

解：

- 1) MMU 遍历页表后发现 0x0 没有有效的页表项，立即触发一个缺页异常，并将控制权交给内核。内核的缺页异常处理程序在进程的 VMA 列表中查找导致异常的地址 0x0，发现不属于任何已分配的合法区域，因此判定这是一次非法访问。
- 2) 当 fork() 被调用时，内核并不会立即复制父进程的物理内存页，而是让子进程的页表项指向与父进程相同的物理页，并将这些共享页在父子进程的页表中都标记为只读。当任何一个进程尝试写入这些共享页时，MMU 会因权限冲突而触发一个缺页异常。内核的异常处理程序捕获此异常后，才会为写入方分配一个新的物理页，将共享页的内容复制过去，然后更新其页表项指向这个新页面并恢复可写权限。
- 3) • 优化页替换算法：采用更智能的页替换算法，可以更准确地预测哪些页面在未来最不可能被访问，换出这些页面，从而有效降低缺页率，减少访问磁盘的次数。

- 预取与聚类写：当发生缺页时，操作系统可以采用预取策略，即猜测程序接下来可能会访问相邻的页面，并提前将它们从磁盘读入内存。同时，在换出页面时，可以将多个脏页聚类在一起，通过一次大的 I/O 操作将它们批量写入磁盘，以提高磁盘利用效率。
- 使用内存压缩：在将页面换出到慢速的磁盘之前，可以先尝试将其在内存中进行压缩，从而用 CPU 时间换取了对磁盘的访问，降低了总体延迟。

17. AArch64 的虚拟内存机制支持包括 4KB 在内的不同的页面大小，通常采用 4 级页表，请回答以下问题：

- 1) AArch64 中的 `ttbr0_el1` 和 `ttbr1_el1` 的作用是什么？对于一个虚拟地址硬件如何决定采用哪一个寄存器指向的页表进行地址翻译？（4 分）
- 2) AArch64 通常支持 4K、2M、1G 三种不同大小的页面，为何页面大小每增加一次是原来的 2^9 倍？使用大页的好处是什么？（4 分）
- 3) 如果映射虚拟地址地址范围 `0x00000000~0x01000000`，则每一级页表分别需要多少个页表页（仅考虑 4K 页）？（4 分）

解：

- 1) `ttbr0_el1` 和 `ttbr1_el1` 都是指向 0 级页表基地址的寄存器。`ttbr0_el1` 用于翻译低地址空间（通常分配给用户进程），而 `ttbr1_el1` 用于翻译高地址空间（通常由内核使用）。硬件通过检查虚拟地址的第 63~48 位来决定使用哪个寄存器。如果全为 0，则它属于低地址空间，硬件会自动使用 `ttbr0_el1`；如果全为 1，则它属于高地址空间，硬件会自动使用 `ttbr1_el1`。
- 2) 原因是：AArch64 的页表结构中，每一级页表都使用 9 位虚拟地址作为索引来查找 512 个页表项中的一个。当使用大页时，意味着页表遍历提前一个级别终止，并将原本用于下一级页表索引的 9 位地址转而用作页内偏移，因此页面大小就扩大了 2^9 倍。

使用大页的好处是：

- 减少 TLB Miss：一个 TLB 条目可以映射更大的内存区域，显著提高了 TLB 的命中率，从而减少了因 TLB Miss 导致的耗时页表遍历，提升了性能。
- 降低内存开销：使用一个大页的页表项可以替代一整张下一级的页表页，从而节省了存储页表自身所需的内存。同时减少了页表的级数，提升了遍历页表的效率。

- 3) 1 个 0 级页表页、1 个 1 级页表页、1 个 2 级页表页、8 个 3 级页表页。

18. 物理内存管理会涉及伙伴系统以及 SLAB 分配器，请回答以下问题：

- 1) 一个大小为 32K 的物理块，其物理地址为 `0x20000000`，则其伙伴块的物理地址是多少？（3 分）
- 2) 操作系统中，伙伴系统和 SLAB 分配器的作用有什么区别？两者在操作系统运行过程中是如何配合的？（3 分）

解:

1) 0x20008000。

2) 两者作用的区别是:

- 伙伴系统是页级物理内存管理器。它的主要职责是管理和分配大块的、物理上连续的内存，分配的单位是页或页的 2 的幂次方倍。它旨在解决外部碎片问题。
- SLAB 分配器是对象级内存管理器。它专注于高效地分配和释放内核中频繁使用的小型数据结构。它通过对象缓存来消除内部碎片，并避免了重复初始化对象的开销。

当 SLAB 需要更多内存来创建其缓存的对象时，它会向底层的伙伴系统申请一个或多个完整的、连续的物理页。伙伴系统满足其请求后，SLAB 分配器再将这些大块的内存页细分成许多特定大小的小对象，以供内核使用。当一个 SLAB 缓存中的整页都变为空闲时，SLAB 分配器可以将该页归还给伙伴系统，以便伙伴系统将其用于其他目的。

19. 系统采用**时钟页置换算法**，假设系统共有 4 个物理内存页，且物理内存页初始状态均为空闲。进程 P 访问页号的序列为 0、1、2、7、0、5、3、5、0、2、7、6，请列出**每次访存**后物理内存中的数据页内容（12 分），并给出上述访存序列一共会产生多少次缺页异常？（3 分）

访存:	0	1	2	7	0	5	3	5	0	2	7	6
访存后的物理内存数据												
缺页												

解:

访存:	0	1	2	7	0	5	3	5	0	2	7	6
物理内存 (页帧 0)	0*	0*	0*	0*	0*	5*	5*	5*	5*	5*	7*	7*
物理内存 (页帧 1)		1*	1*	1*	1*	1	3*	3*	3*	3*	3	6*
物理内存 (页帧 2)			2*	2*	2*	2	2	2	0*	0*	0	0
物理内存 (页帧 3)				7*	7*	7	7	7	7	2*	2	2
缺页	是	是	是	是	否	是	是	否	是	是	是	是

共产生 10 次缺页异常。